I want to mod Rain World but I don’t know how to code!!!



If this sounds familiar, you’re not alone. That was (and still is) me, among many other Rain Worlders. This document aims to help you jumpstart your journey by organising information.

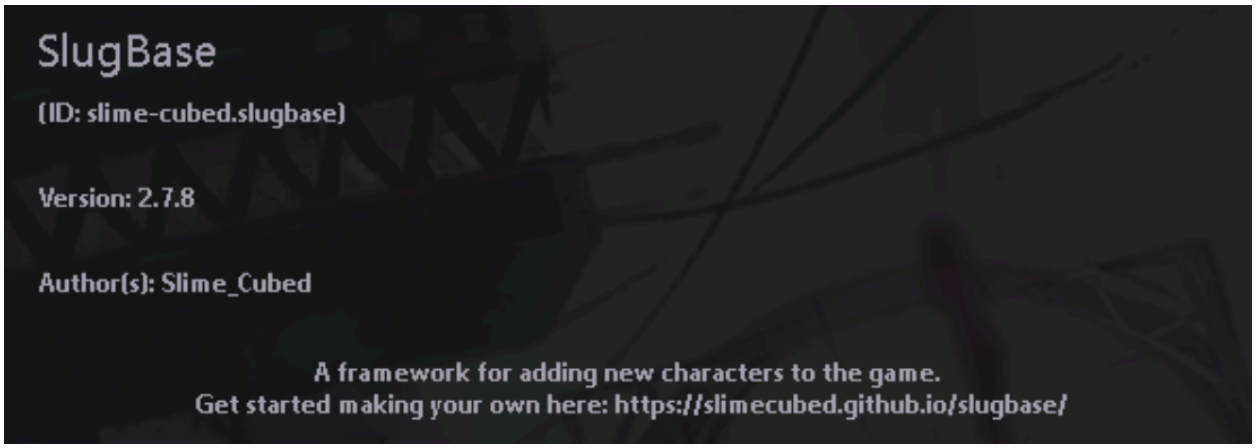
Disclaimer: this document is written based on my understanding. I do not claim to be a professional coder or modder by any means, and much of this document is written like a blog. I hope this format will help newbie modders through insight of problems and roadblocks I ran into, but this is by no means a comprehensive list.

So. Modding without coding. Introducing your new best friend: **Slugbase**. Slugbase is exactly what it sounds like, a template (written by SlimeCubed) that allows you to create new custom slugcats more easily. This includes custom colours, stats, cutscenes, dreams, art etc. It has its own documentation from installation to usage here: https://slimecubed.github.io/slugbase/articles/template.html

Slugbase Features

What exactly are you renaming, though? What do all the files that you just messed with do? Generally: xxx**.cs** represents a C# file. Any code should go in a file with this ending. xxx**.json** represents something like a text file. Only strings ("words/phrases in colons like this"), numbers (-5, 2, 0.26, 3.14159, …), and bools (true/false) should go in a file with this ending.

Extras.cs: you'll find this file under your mod, but it's never mentioned. It's part of the setup process (that you can see in the Rain World wiki) that's been done up for you. As it said, don't mess with it
Plugin.cs: where you can register and define all your hooks (like the 3 examples given!)
modinfo.json: turns up as information on the mod page within Rain World itself. In order shown below: name, id, version, authors, description, (requirements is out of screenshot but is listed below the thumbnail in-game)



the_prototype.json: features applied to your custom slugcat! You can find a full list of configurable, built-in options here: https://slimecubed.github.io/slugbase/articles/features.html?tabs=slugcatname . Name and description will turn up on the slugcat select screen.
Notable features:

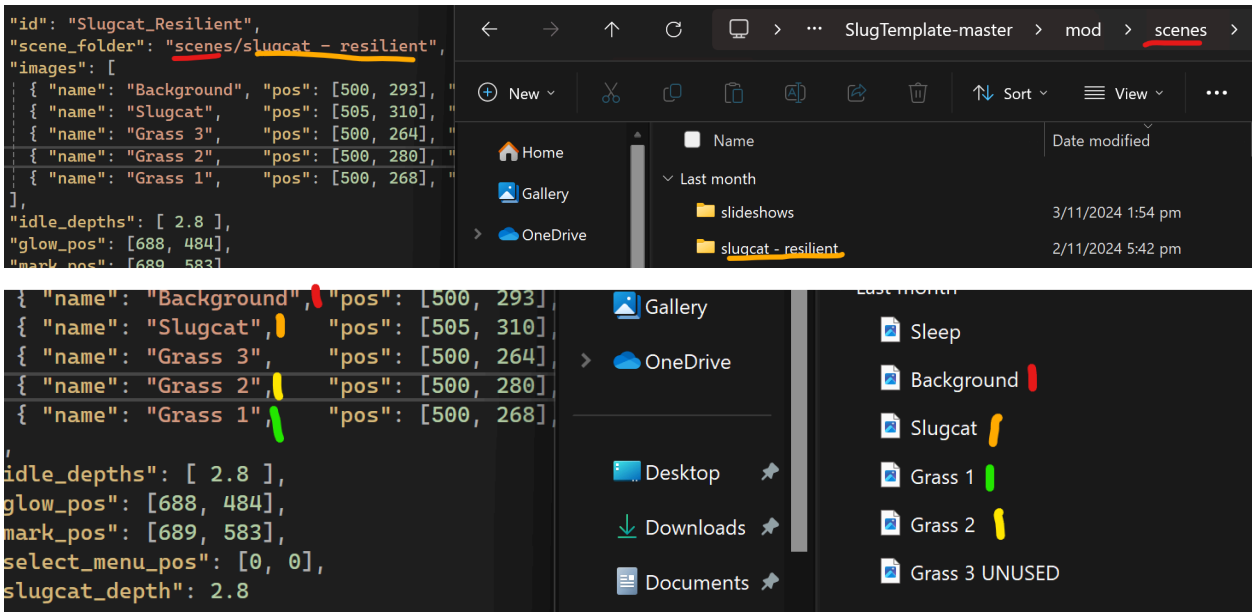
- "colors" changes the colour of slugcat's body in-game only vs "custom_colors" which can change multiple aspects (ie body and eyes) as well as dictate custom arena colours
- any kind of speed modifier does not affect any movement tech (eg sliding, rolling)
- "start_room" allows you to change the start room of the slugcat but not the spawn point, so you're likely going to end up stuck in a wall. Changing the spawn point requires a hook
- for "select_menu_screen", "select_menu_screen_ascended", "sleep/starve/death_scene", "intro/outro_slideshow", you will require an additional

.json file *each* detailing it. Select/ascended has been done for you (see below), the rest has an in-depth tutorial under “getting started” in Slugbase docs

slugcat_prototype.json: the image on the slugcat select screen

ghost_prototype.json: the image on the slugcat select screen (if ascended)

-> to note both of these files are just telling the program *what* to do with the images provided! Actual images should be uploaded to correct folders (whatever you cited in the .json file). Name everything appropriately



-> “grass 3” is listed to appear in the .json but I do not have an image for it, therefore only Background, Slugcat, Grass 2 and Grass 1 loads. I left Grass 3 listed in the .json as I am planning to add it in at a later date. This does not break the game, don’t worry

Custom Skins

You'll be pleased to know that there is another easy way out that requires only your arting skills and not your coding skills: **Dress My Slugcat** (DMS).

DMS has its own Discord server: https://discord.gg/gRf6pMcuRp as well as detailed documentation: https://steamcommunity.com/sharedfiles/filedetails/?id=2902555797 so I won't cover this.

The second method involves creating visual hooks within the code itself that informs the game how to change the graphics of the slugcat. To update when I learn how to do this

This is all well and good.. But I want to add something else to my slugcat! Something special, something unique not covered here-

I hate to say it this way but to say it this way: there's no going around it. You're going to have to learn coding. C#, to be specific, using object oriented programming. You're going to have to wade through the horror that is the original Rain World code and make sense of it. You're going to need to think critically from now on- no more following step-by-step instructions.



Yay! Yippee, hi, hi hi hi! Woohoooo! Yippee hi hello how are you I hope you enjoy your stay. Welcome to hell.

If you couldn't tell I don't love this part- but it gets easier as you keep going.

I'll be linking a lot of videos and external resources after this point (which are the original videos/resources I myself used/can recommend). To put it simply, you can't go to sleep thinking of code and wake up familiar with it: if you are new to coding, you're going to have to learn that logic, and if you aren't, you're going to have to get familiar with that syntax. It doesn't matter either way. Some good old practice will fix that.

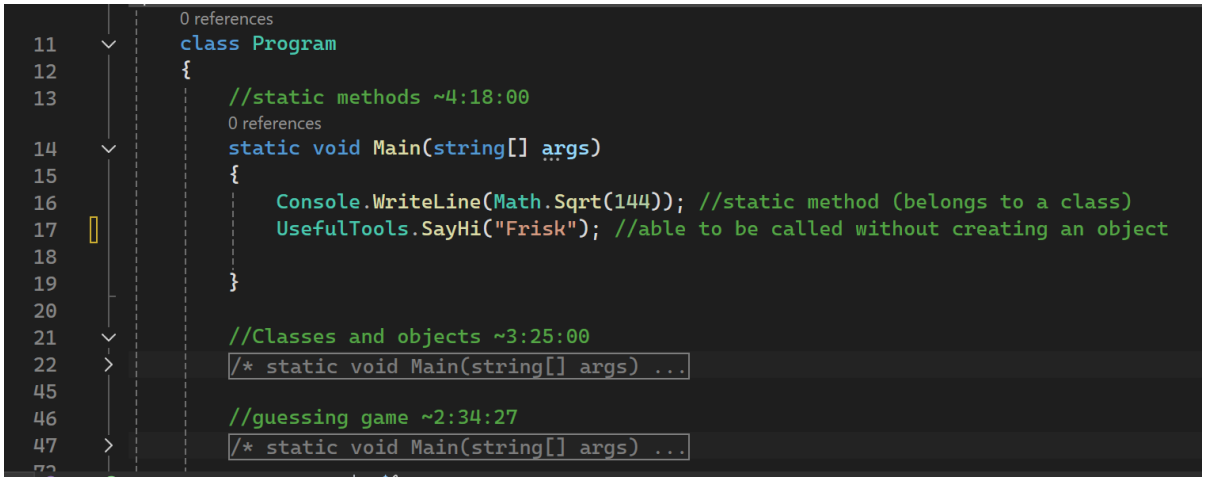
Basic Coding

Both of these are from freeCodeCamp.org (a YouTube channel). They both go over how to code from basic syntax to coding logic, have clear, well explained examples, and step by step breakdowns of what they are doing, what you can expect to see etc. The second one is longer (by around 2x) but as a result of the extra content you can expect to see more content covered, and it also goes further into advanced concepts.

Personally, I feel the first video goes at a much slower pace in general, with a lot more in depth explanations, so I would recommend watching that first especially if you're very new! You can always watch the second video after and either skip over the parts you already know, or clock in a second round of practice. I would also highly recommend coding along with them, crack out Visual Studio if you don't already have it and follow along.

Write notes (//commenting like this is a great way to do so, without Visual Studio running into errors thinking it's code!), practice code, pause the video and try your hand at coding it yourself then correct your mistakes. It gets easier with time, effort, and practice, and slowly the monstrous chunk of pure nonsense that is Rain World's code dissolves into something more readable.

To note that, if you are using Visual Studio 2022 the layout of the basic structure may be slightly different (most notably when trying to create a class, the basic startup code). It won't be exactly like the video, which uses an older version of Visual Studio, but it still works, there is no need to change anything to fit the videos exactly as shown :)



Video 1 (4h 31min): https://youtu.be/GhQdlIFylQ8?si=PBK_Ua-zZSUfOc5Z
Video 2 (8h 17min): https://youtu.be/YrtFtdTTfv0?si=boZpAQhH-YDIOMR

Object Oriented Programming (OOP)

A type of coding logic that classifies code into both classes and objects, to make them easier to work with. If you have already watched the previous video(s), or are familiar with C#, this should sound familiar!

The following are my notes based off this video:
https://youtu.be/SiBw7os-_zI?si=o8kN2hUgopxUzHz3

Classes: “templates” of objects eg knight class in chess
Objects: specific instances of classes eg white knight 1

1. Encapsulation
    - Bundle data + methods that use those data together in a class
    - Hide data in a class, preventing anything outside of the class from interacting with it
    - Getting (obtaining information) and setting (changing information) methods
    - Keeps programmer in control of data
2. Abstraction
    - Only showing essential details (interface), not inner workings (implementation)
    - Users should know how to operate without knowing why it works
3. Inheritance
    - Allows subclasses to derive from superclasses
    - Access modifiers: public (value changeable from within and outside class), private (value changeable from within class only), protected (value changeable from within class + subclasses only)
4. Polymorphism
    - A method that is able to take on multiple forms
    - Dynamic:
        - A method has the same name and arguments in both superclass and subclass
        - Subclass method overrides superclass method
        - Allows superclass method to be written without if-elses catering for subclasses
    - Static:
        - A method has the same name but different arguments in the same class; “method overloading”
        - Different parameters, parameters in different orders
        - Be careful to call correct method

Miscellaneous Syntax

You may have noticed, after the 40 hours a day you've poured into learning coding, you *still* don't recognise certain functions in the Rain World code.. what is Mathf.Lerp(-90f, 90f, UnityEngine.Random.value) supposed to mean, anyway????

Now, as you know, in C# the syntax goes something like this: Class.Method(arguments, of, method). Mathf is a Unity Class that, you guessed it, contains math functions. Anything from maximum to minimum to linear interpolation. Give it a search! Unity has its own docs too (here I have linked the Mathf page): https://docs.unity3d.com/6000.0/Documentation/ScriptReference/Mathf.html

Another common thing you may have seen in repositories of modded code